



ICpedia Internship Program

Physical Design (PnR) Team

Final Assignment

Name: Ahmed Alaa Mohamed

University: Ain Shams University

Status: Fresh Graduate (Class of 2026)

Submission Date: September 12, 2026

Contents

Introduction	1
Initial Setup and Starting Frequency	2
Synthesis Configuration	2
Synthesis Exploration	2
Floorplan Configuration	4
Placement Configuration	4
CTS Configuration	5
Routing Configuration	5
Max Frequency Trials	5
Min Area With Threshold Frequency Trials	7
Pin Placement Configuration	8
Power Distribution Network (PDN) Configuration	9
Metal 1 (Standard Cell Power Rails)	9
Metal 2	10
Metal 3	11
Metal 4	11
Metal 5	12
Combined PDN	12
Placement and CTS Optimization	13
Routing Configuration: DRT_OPT_ITERS Tradeoff	13
GDS and LEF Comparison in KLayout	13
Metal Layer Validation in KLayout	16
LEF Dimension Validation	18
Full Routed Chip	20
Extra Automation Section	20
Integration into flow.tcl	22
Report Summarization	23
Log Parsing	27
Liberty (.lib) Parsing for Area Normalization	28
Gate-Level Netlist Parsing	28
Results Summary	29
Conclusion	29

Introduction

This report documents the RTL2GDSII implementation of the **wbqspiflash** design, completed as the final assignment of the ICpedia Internship Program, Physical Design (PnR) Team, using the OpenLane/OpenROAD toolchain on the sky130 process, covering the full flow from RTL synthesis to sign-off.

The project follows a two-stage optimization target: first reaching the **maximum achievable clock frequency** under relaxed area constraints, then, once this frequency is fixed, **minimizing die area** by tuning placement density and CTS parameters without violating timing or design rules.

Beyond the core PnR steps, the project also involved custom pin ordering, a power distribution network spanning met1–met5, and post-layout verification of the GDSII/LEF views in KLayout, including metal layer mapping validation against the PDK's stream/layer definition file.

Python automation was used at two levels to manage the large number of configurations explored: OpenLane's `run_designs.py` for sweeping `CLOCK_PERIOD` and placement/CTS parameters across runs, and a custom report-parsing script for extracting frequency, area, timing (WNS/TNS), and DRC metrics for trial comparison. The extra Python-based report/log parsing automation developed for this project is available in my own repository: [Python-ASIC-Flowscope](#).

Initial Setup and Starting Frequency

Synthesis Configuration

```
#synthesis configurations
set ::env(CLOCK_PERIOD) "10"
set ::env(SYNTH_STRATEGY) "AREA 3"
set ::env(SYNTH_CAP_LOAD) 5
set ::env(SYNTH_CLOCK_UNCERTAINTY) 0.1
set ::env(SYNTH_CLOCK_TRANSITION) 0.1
set ::env(SYNTH_BUFFERING) 1
set ::env(SYNTH_SIZING) 0
set ::env(SYNTH_NO_FLAT) 0
set ::env(SYNTH_MAX_FANOUT) 30
set ::env(CLOCK_BUFFER_FANOUT) 16
set ::env(IO_PCT) 0.2
set ::env(SYNTH_TIMING_DERATE) 0.00
```

Figure 1: Synthesis configuration variables used in sky130A_sky130_fd_sc_hd_config.tcl.

For the clock period, the value was swept downward until the minimum feasible period was identified, and the synthesis strategy was set to **AREA 3**, selected based on the synthesis exploration results. The capacitive load (**SYNTH_CAP_LOAD**) was decreased, and the clock uncertainty was set to 10% of the clock period, which is a reasonable assumption since many PLLs exhibit around 10% jitter/uncertainty. The timing derate (**SYNTH_TIMING_DERATE**) was set to 0 to remove additional pessimism from the timing analysis and allow the maximum achievable frequency to be identified without being masked by derating margins. The maximum fanout (**SYNTH_MAX_FANOUT**) was also increased, as an earlier run with a lower value resulted in fanout violations during synthesis.

Synthesis Exploration

To select the most suitable synthesis strategy for this design, OpenLane's built-in synthesis exploration utility was used. This automated, Python-driven feature runs the synthesis step against all available strategies (**DELAY0-DELAY4** and **AREA0-AREA3**) and reports the resulting gate count, area, and delay for each, allowing an informed choice before proceeding through the rest of the flow. It was invoked using the following command:

```
1 ./flow.tcl -design wbqspiflash -synth_explore
```



Figure 2: Synthesis strategies comparison: Area vs. Delay across all explored strategies.

Best Area	Best Gate Count	Best Delay
12202.95	1450.0	2403.7
AREA2	AREA2	AREA3

Strategy	Gate Count	Area (um ²)	Delay (ps)	Gates Ratio	Area Ratio	Delay Ratio
DELAY0	1732.0	15376.0	4961.2	1.194	1.26	2.063
DELAY1	1580.0	14052.23	4655.67	1.089	1.151	1.936
DELAY2	1572.0	13998.43	4488.45	1.084	1.147	1.867
DELAY3	1574.0	14060.99	4401.15	1.085	1.152	1.83
DELAY4	1627.0	14025.95	3025.87	1.122	1.149	1.258
AREA0	1489.0	12618.35	4695.68	1.026	1.034	1.953
AREA1	1555.0	12922.39	5201.8	1.072	1.058	2.164
AREA2	1450.0	12202.95	7088.46	1.0	1.0	2.948
AREA3	1992.0	15926.52	2403.7	1.373	1.305	1.0

Synthesis Strategies Comparison

Figure 3: Detailed synthesis exploration results: gate count, area, delay, and their respective ratios for each strategy.

As shown in Figures 2 and 3, AREA3 achieved the best (lowest) delay of 2403.7 ps among all strategies, at the cost of a larger area (15 926.52 μm^2) and higher gate count (1992) compared to the other options. Since the first stage of this project targets the maximum achievable clock frequency under relaxed area constraints, AREA3 was selected as the synthesis strategy to obtain the fastest possible timing. Area was then optimized in later stages, once the target frequency was fixed, without changing the synthesis strategy.

Floorplan Configuration

```
# floorplan configurations
set ::env(FP_SIZING) relative
set ::env(FP_CORE_UTIL) 40
set ::env(FP_CORE_MARGIN) 0
set ::env(FP_ASPECT_RATIO) 1
set ::env(FP_PDN_AUTO_ADJUST) 0
set ::env(FP_PDN_VOFFSET) 0
set ::env(FP_PDN_HOFFSET) 0
set ::env(FP_PDN_VPITCH) 30
set ::env(FP_PDN_HPITCH) 30
set ::env(FP_PDN_CORE_RING) 0
set ::env(FP_PDN_LOWER_LAYER) met2
set ::env(FP_PDN_UPPER_LAYER) met5
set ::env(FP_PDN_RAILS_LAYER) met1
set ::env(FP_PIN_ORDER_CFG) $::env(DSIGN_DIR)/pin_order.cfg
set ::env(FP_IO_VTHICKNESS_MULT) 1
set ::env(FP_IO_HTHICKNESS_MULT) 1
set ::env(FP_IO_VLENGTH) 2
set ::env(FP_IO_HLENGTH) 2

sky130A_sky130_fd_sc_hd_config.tcl [R0]
```

Figure 4: Floorplan configuration variables used in sky130A_sky130_fd_sc_hd_config.tcl.

The floorplan sizing mode (FP_SIZING) was set to `relative`, with the core utilization (FP_CORE_UTIL) swept throughout the flow to reach the minimum achievable die area, as discussed in the frequency and area exploration sections, while the core margin was left at 0.

For the PDN, FP_PDN_AUTO_ADJUST was disabled and the offsets set to 0, with matching vertical/horizontal pitches of 30, a combination found to center the PDN stripe pattern correctly within the core. The core ring was disabled, and the grid was configured across `met1` (rails), `met2` (lower stripe layer), and `met5` (upper stripe layer).

Custom pin ordering (FP_PIN_ORDER_CFG) was pointed to `pin_order.cfg`. The I/O pin thickness and length multipliers (FP_IO_VTHICKNESS_MULT, FP_IO_HTHICKNESS_MULT = 1, FP_IO_VLENGTH, FP_IO_HLENGTH = 2) were set per the project requirements and left unchanged for the rest of the flow.

Placement Configuration

```
#placement configurations
#set ::env(PL_TARGET_DENSITY) 0.46
set ::env(PL_ROUTABILITY_DRIVEN) 1
set ::env(PL_TIME_DRIVEN) 1
set ::env(PL_RESIZER_DESIGN_OPTIMIZATIONS) 1
set ::env(PL_RESIZER_TIMING_OPTIMIZATIONS) 1
set ::env(PL_RESIZER_HOLD_MAX_BUFFER_PERCENT) 50
set ::env(PL_RESIZER_SETUP_MAX_BUFFER_PERCENT) 50
```

Figure 5: Placement configuration variables used in sky130A_sky130_fd_sc_hd_config.tcl.

The placement target density (PL_TARGET_DENSITY) was swept across multiple values to identify the minimum achievable area at the threshold frequency while still maintaining a clean DRC report. The resizer setup buffer percentage (PL_RESIZER_SETUP_MAX_BUFFER_PERCENT) was also varied in an attempt to further increase the

achievable frequency. Routability-driven placement (PL_ROUTABILITY_DRIVEN) and time-driven placement (PL_TIME_DRIVEN) were both enabled to account for routing congestion and timing during placement, while all remaining placement variables were left at their default values.

CTS Configuration

```
# clock tree synthesis configurations
set ::env(CLOCK_TREE_SYNTH) 1
set ::env(CTS_TARGET_SKEW) 200
set ::env(CTS_TOLERANCE) 200
set ::env(CTS_REPORT_TIMING) 1
set ::env(CTS_SINK_CLUSTERING_SIZE) 25
set ::env(CTS_SINK_CLUSTERING_MAX_DIAMETER) 50
sky130A_sky130_fd_sc_hd_config.tcl [+]
```

Figure 6: Clock tree synthesis configuration variables used in sky130A_sky130_fd_sc_hd_config.tcl.

Clock tree synthesis was enabled (CLOCK_TREE_SYNTH), and different values of CTS_TOLERANCE were tried in order to achieve a higher quality of results (QoR), alongside the corresponding CTS_TARGET_SKEW setting. Timing reporting after CTS (CTS_REPORT_TIMING) was enabled to directly observe the impact of each configuration on setup and hold timing. The sink clustering size and maximum diameter (CTS_SINK_CLUSTERING_SIZE, CTS_SINK_CLUSTERING_MAX_DIAMETER) were also set to control how clock sinks are grouped during clustering, balancing skew against clock tree buffer count.

Routing Configuration

```
# Routing configurations
set ::env(GLOBAL_ROUTER) fastroute
set ::env(DETAILED_ROUTER) tritonroute
set ::env(GLB_OPTIMIZE_MIRRORING) 1
set ::env(DRT_OPT_ITERS) 64
set ::env(GLB_RESIZER_TIMING_OPTIMIZATIONS) 1
set ::env(GLB_RESIZER_HOLD_MAX_BUFFER_PERCENT) 50
set ::env(GLB_RESIZER_SETUP_MAX_BUFFER_PERCENT) 50
```

Figure 7: Routing configuration variables used in sky130A_sky130_fd_sc_hd_config.tcl.

fastroute was used as the global router and tritonroute as the detailed router, with global routing mirroring (GLB_OPTIMIZE_MIRRORING) enabled to reduce via count and improve routability. The detailed routing optimization iteration count (DRT_OPT_ITERS) was set to 64, explored further as a runtime/QoR trade-off in the following section.

Timing-driven resizer optimizations during global routing (GLB_RESIZER_TIMING_OPTIMIZATIONS) were enabled, with both the hold and setup maximum buffer percentages (GLB_RESIZER_HOLD_MAX_BUFFER_PERCENT, GLB_RESIZER_SETUP_MAX_BUFFER_PERCENT) set to 50, allowing the resizer sufficient buffering budget to close any remaining timing violations introduced by routing parasitics.

Max Frequency Trials

Table 1 summarizes the clock period sweep (CLOCK_PERIOD = 8, 7, 6, 5 ns) used to identify the maximum achievable frequency under relaxed area constraints. Timing results use MCA (Multi-Corner Analysis) worst-case values across all corners.

Trial	Clock Period	Frequency	Area (μm^2)	Setup (WNS/TNS)	Hold (WNS/TNS)
No.1	8 ns	125 MHz	49581	0.00 / 0.00	0.17 / 0.00
No.2	7 ns	143 MHz	49575	0.00 / 0.00	0.18 / 0.00
No.3	6 ns	167 MHz	49604	-0.68 / -30.14	0.17 / 0.00
No.4	5 ns	200 MHz	49568	-1.55 / -168.72	0.17 / 0.00

Table 1: Maximum frequency exploration trials.

Trials No.1 and No.2 (8 ns and 7 ns) show clean timing across all corners with zero DRC violations. From No.3 (6 ns) onward, setup timing fails at the Slowest corner, worsening as the period is further reduced to No.4 (5 ns). Hold timing remains clean across all four trials. Based on these results, the maximum achievable frequency lies between the 6 ns and 7 ns clock periods, since 7 ns closes cleanly while 6 ns already violates setup timing.

To narrow this range down further, a second design exploration run was performed. This was done using OpenLane's built-in design exploration utility, `run_designs.py`, with multiple threads enabled to run several configurations in parallel and save runtime. The following regression configuration was written for this sweep:

```
1 CLOCK_PERIOD=(8,7,6,5)
```

7 ns was selected as the threshold frequency for the area-minimization phase that follows, pending the results of the finer sweep between 6 ns and 7 ns.

Table 2 summarizes the second, finer design exploration run performed between 6.35 ns and 6.5 ns, narrowing down the maximum achievable frequency identified from the first sweep.

Trial	Clock Period	Frequency	Area (μm^2)	Setup (WNS/TNS)	Hold (WNS/TNS)
No.5	6.5 ns	153.85 MHz	49588	0.00 / 0.00	0.17 / 0.00
No.6	6.45 ns	155.04 MHz	49504	0.00 / 0.00	0.18 / 0.00
No.7	6.4 ns	156.25 MHz	49558	-0.16 / -2.15	0.15 / 0.00
No.8	6.35 ns	157.48 MHz	49595	-0.08 / -0.89	0.17 / 0.00

Table 2: Second design exploration: fine clock period sweep.

Trials No.5 and No.6 (6.5 ns and 6.45 ns) close cleanly with no setup or hold violations across all corners. From No.7 (6.4 ns) onward, setup timing violates at the Slowest corner. Hold timing remains clean across all four trials. Based on this finer sweep, the maximum achievable frequency lies between **6.4 ns and 6.45 ns**, with 6.45 ns being the smallest clean clock period identified across both exploration runs.

Table 3 summarizes the third, even finer design exploration run performed between 6.41 ns and 6.44 ns, further narrowing down the maximum achievable frequency identified from the second sweep.

Trial	Clock Period	Frequency	Area (μm^2)	Setup (WNS/TNS)	Hold (WNS/TNS)
No.9	6.44 ns	155.28 MHz	49499	0.00 / 0.00	0.16 / 0.00
No.10	6.43 ns	155.52 MHz	49521	0.00 / 0.00	0.18 / 0.00
No.11	6.42 ns	155.76 MHz	49575	-0.04 / -0.09	0.18 / 0.00
No.12	6.41 ns	156.01 MHz	49515	-0.10 / -1.17	0.17 / 0.00

Table 3: Third design exploration: finest clock period sweep.

Trials No.9 and No.10 (6.44 ns and 6.43 ns) close cleanly with no setup or hold violations across all corners. From No.11 (6.42 ns) onward, setup timing violates at the Slowest corner. Hold timing remains clean across all four trials. In conclusion, the optimum clock period lies between **6.43 ns and 6.42 ns**, with **6.43 ns** identified as the smallest clean clock period across all three exploration runs, and therefore selected as the final threshold frequency for the subsequent area-minimization phase.

Table 4 summarizes the fourth, final refinement of the clock period sweep, performed between 6.422 ns and 6.429 ns to pinpoint the exact maximum achievable frequency.

Trial	Clock Period	Frequency	Area (μm^2)	Setup (WNS/TNS)	Hold (WNS/TNS)
No.13	6.429 ns	155.55 MHz	49576	0.00 / 0.00	0.17 / 0.00
No.14	6.427 ns	155.60 MHz	49514	0.00 / 0.00	0.17 / 0.00
No.15	6.425 ns	155.64 MHz	49559	0.00 / 0.00	0.17 / 0.00
No.16	6.422 ns	155.72 MHz	49589	0.00 / 0.00	0.17 / 0.00

Table 4: Fourth design exploration: final clock period refinement.

All four trials (No.13–No.16) close cleanly with no setup or hold violations across all corners and zero DRC violations. Since every value in this range is clean, the smallest period tested, **6.422 ns**, represents the maximum achievable frequency for this design, and is therefore selected as the final threshold frequency for the subsequent area-minimization phase.

Trial	Utilization	DENSITY	Area (μm^2)	Setup (WNS/TNS)	Hold (WNS/TNS)
No.17	42%	0.45	47121	-0.03 / -0.05	0.17 / 0.00
No.18	41%	0.46	48381	-0.01 / -0.01	0.18 / 0.00
No.19	43%	0.48	46097	-0.25 / -4.25	0.17 / 0.00
No.20	41%	0.45	48269	0.00 / 0.00	0.17 / 0.00

Table 5: Minimum area exploration trials at the fixed 6.422 ns clock period.

Min Area With Threshold Frequency Trials

With the frequency exploration phase complete, the target clock period for this stage was fixed at **10 ns (100 MHz)**. This period was chosen as a comfortably relaxed constant frequency, well outside the timing-critical range identified in the previous section, so that the full effort of this phase could be dedicated to minimizing die area rather than closing timing.

Since timing is no longer the bottleneck at 10 ns, the synthesis strategy was revisited. Referring back to the synthesis exploration results in Figures 2 and 3, **AREA3** (used earlier to obtain the fastest possible delay) is no longer justified: its low delay comes at the cost of a larger gate count and area, which is unnecessary once the frequency target is relaxed to 10 ns. Instead, **AREA0** was selected as the synthesis strategy for this stage, since it produces the smallest area among the explored strategies while its (comparatively worse) delay is still far below the 10 ns threshold and therefore does not impact timing closure.

Table ?? summarizes the trials performed while minimizing area at the fixed 10 ns clock period, sweeping placement utilization and PL_TARGET_DENSITY. Timing results use MCA (Multi-Corner Analysis) worst-case values across all corners.

Trial	Synthesis Strategy	Utilization	PL_TARGET_DENSITY	Area (μm^2)	Setup (WNS/TNS)	Hold (WNS/TNS)
No.1	AREA0	50%	0.55	34476	0.00 / 0.00	0.13 / 0.00
No.2	AREA3	50%	0.55	39547	0.00 / 0.00	0.18 / 0.00
No.3	AREA0	56%	0.61	30604	0.00 / 0.00	0.18 / 0.00
No.4	AREA0	57%	0.62	30234	0.00 / 0.00	0.18 / 0.00

Table 6: Minimum area exploration trials at the relaxed 10 ns clock period.

Trials No.1 and No.2 were run at the same utilization and PL_TARGET_DENSITY (50% / 0.55) to directly compare synthesis strategies under identical placement settings. AREA3 (No.2) results in a larger area ($39\,547\,\mu\text{m}^2$) than AREA0 (No.1, $34\,476\,\mu\text{m}^2$), confirming that the faster, larger-area strategy brings no benefit at this relaxed 10 ns target and that AREA0 is the more efficient choice for this phase. Both trials close cleanly with zero DRC violations.

Having confirmed AREA0 as the synthesis strategy, the placement utilization and target density were then pushed further to reduce area. Trial No.3 (56% / 0.61) reduced the area to $30\,604\,\mu\text{m}^2$, and Trial No.4 (57% / 0.62) reduced it further to $30\,234\,\mu\text{m}^2$, both while maintaining clean setup and hold timing across all corners and zero DRC violations.

Trial No.4 (utilization 57%, PL_TARGET_DENSITY = 0.62, AREA0) achieved the smallest clean area of $30\,234\,\mu\text{m}^2$ and was therefore selected as the optimal configuration for the area-minimization phase at the fixed 10 ns clock period.

In addition to the synthesis strategy and placement density sweeps above, several floorplan-level variables were also explored in an attempt to further reduce the die area: FP_IO_VLENGTH, FP_IO_HLENGTH, BOTTOM_MARGIN_MULT, TOP_MARGIN_MULT, LEFT_MARGIN_MULT, and RIGHT_MARGIN_MULT. These control the I/O pin length and the core-to-die margins on each side of the floorplan. Sweeping these variables across a range of values did not produce any significant change in the final die area, and the dominant factors remained the synthesis strategy and placement target density as shown in Table ???. As a result, these floorplan margin/IO variables were left at their default values for the remainder of this phase, and further area reduction was pursued through placement density alone.

Pin Placement Configuration

Custom pin ordering was applied by editing `/home/opentools/OpenLane/designs/wbqspiflash/pin_order.cfg`, constraining input pins (green) to the East/West sides and output pins (red) to the North/South sides, with `i_wb_data[*]` restricted to West only and `o_wb_data[*]` restricted to North only. The resulting placement was briefly validated in the OpenROAD GUI, as shown in Figure 8.

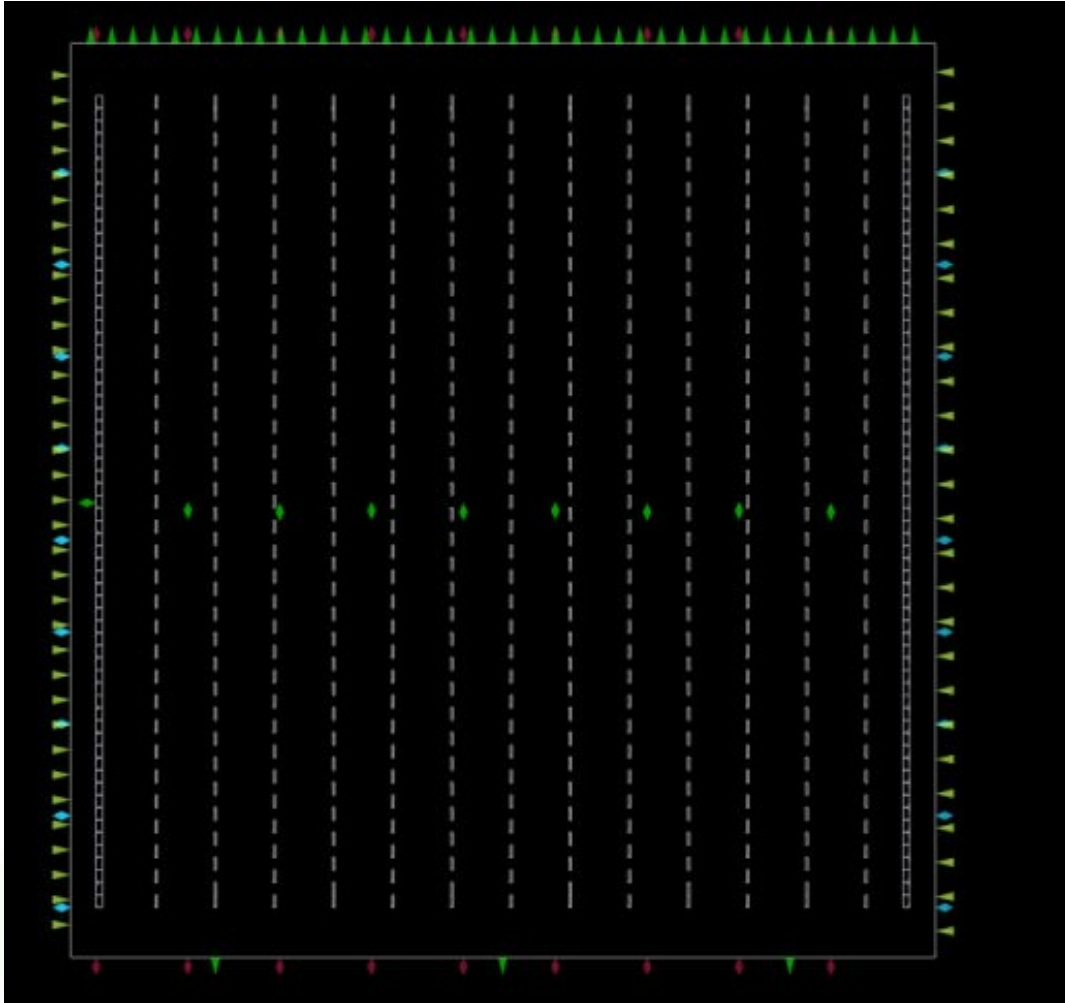


Figure 8: Pin placement validation in the OpenROAD GUI.

Power Distribution Network (PDN) Configuration

The power distribution network was configured to span all metal layers from `met1` to `met5`, using OpenLane's PDN generation script (`pdn.tcl`) with a standard-cell grid defined over the `CORE` voltage domain. The core ring was disabled (`FP_PDN_CORE_RING = 0`), and each metal layer was visually verified individually in the OpenROAD GUI before checking the combined grid.

Metal 1 (Standard Cell Power Rails)

`met1` carries the standard cell power/ground rails, generated with `-followpins` so that a rail is placed along every standard cell row, as shown in Figure 9.

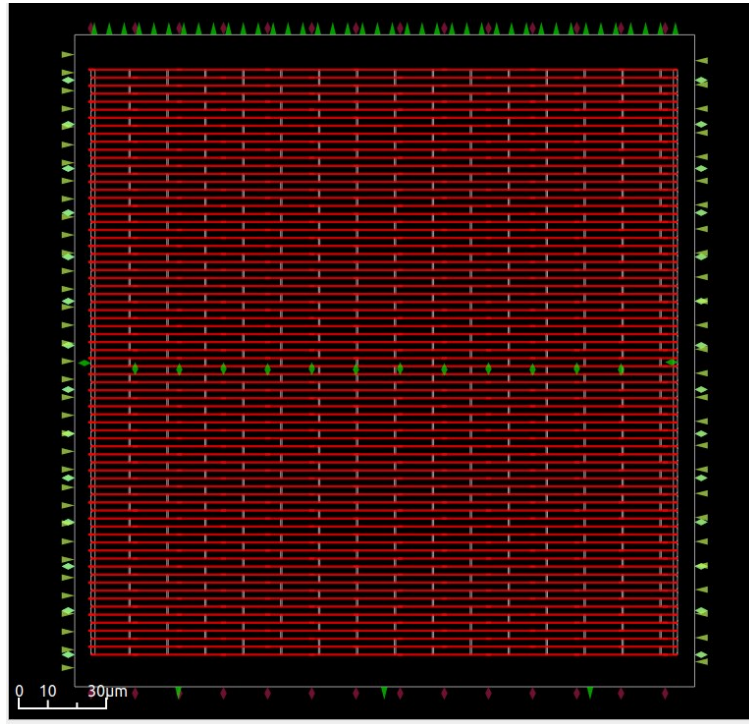


Figure 9: Metal 1 power rails, following the standard cell rows.

Metal 2

met2 corresponds to the vertical stripes of the standard cell grid (FP_PDN_LOWER_LAYER), spaced according to FP_PDN_VPITCH and FP_PDN_VOFFSET, as shown in Figure 10.

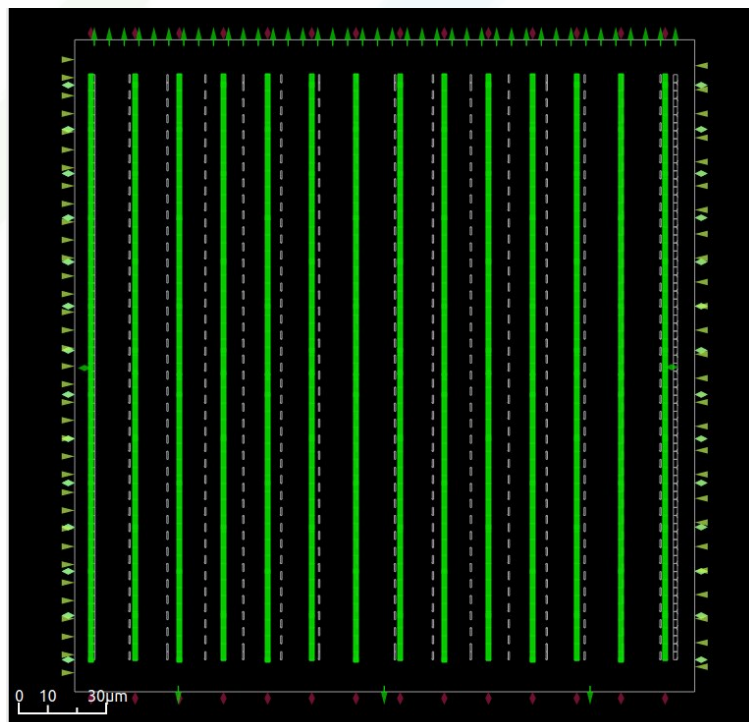


Figure 10: Metal 2 vertical power/ground stripes.

Metal 3

`met3` provides the horizontal PDN stripes; tuning `FP_PDN_HOFFSET` and disabling auto-adjust centered them, fixing the alignment issue (Figure 11).

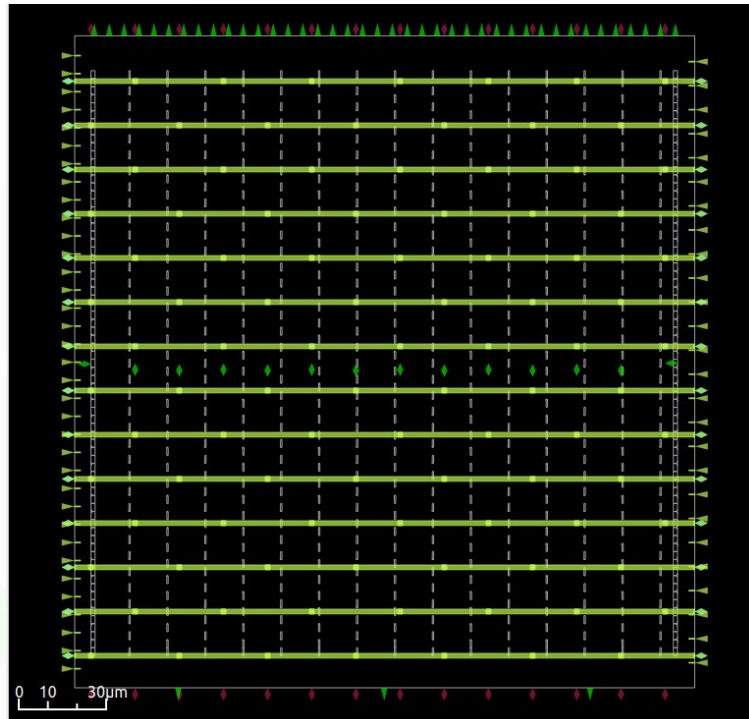


Figure 11: Metal 3 horizontal power/ground stripes.

Metal 4

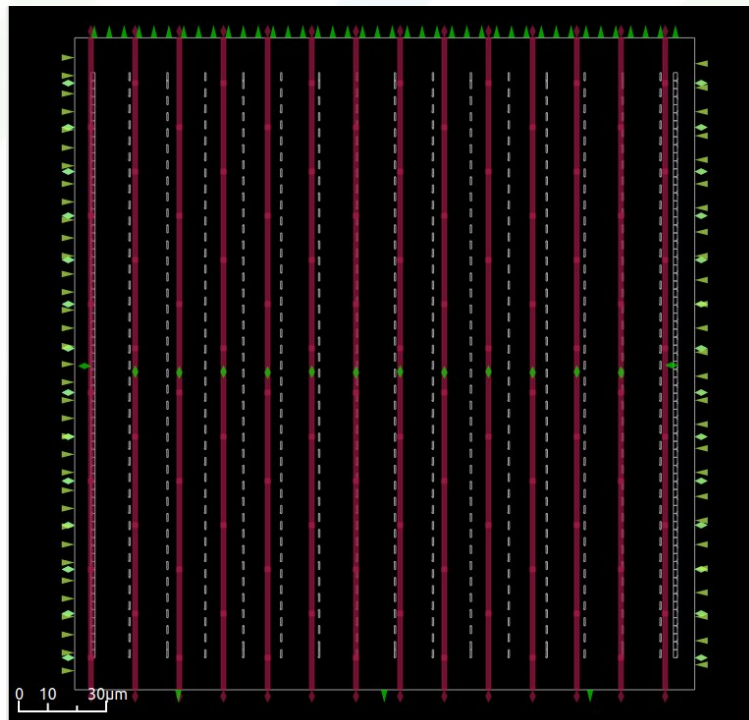


Figure 12: Metal 4 vertical power/ground stripes.

Metal 5

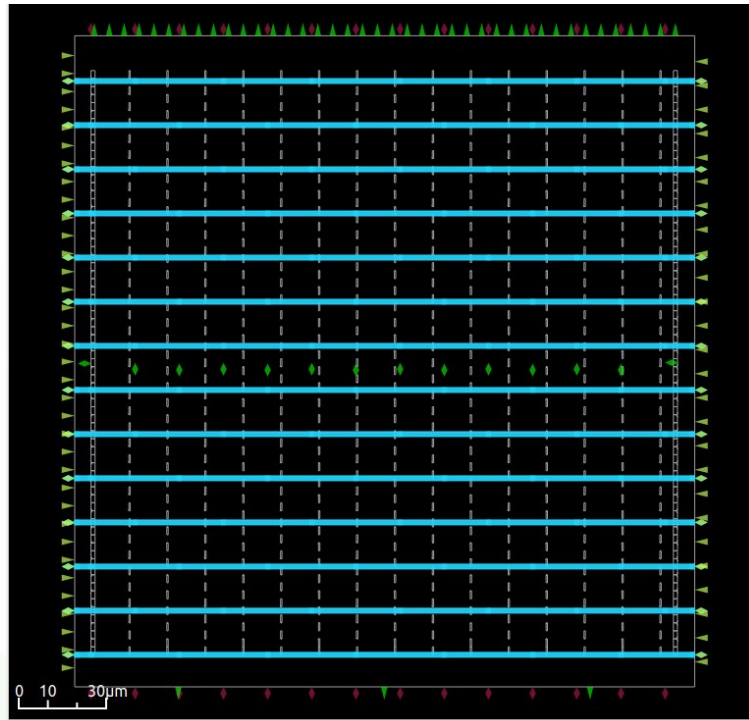


Figure 13: Metal 5 horizontal power/ground stripes.

Combined PDN

Figure 14 shows the full PDN from `met1–met5`, with standard-cell rails, vertical `met2/met4` stripes, and horizontal `met3/met5` stripes forming a connected grid across the core.

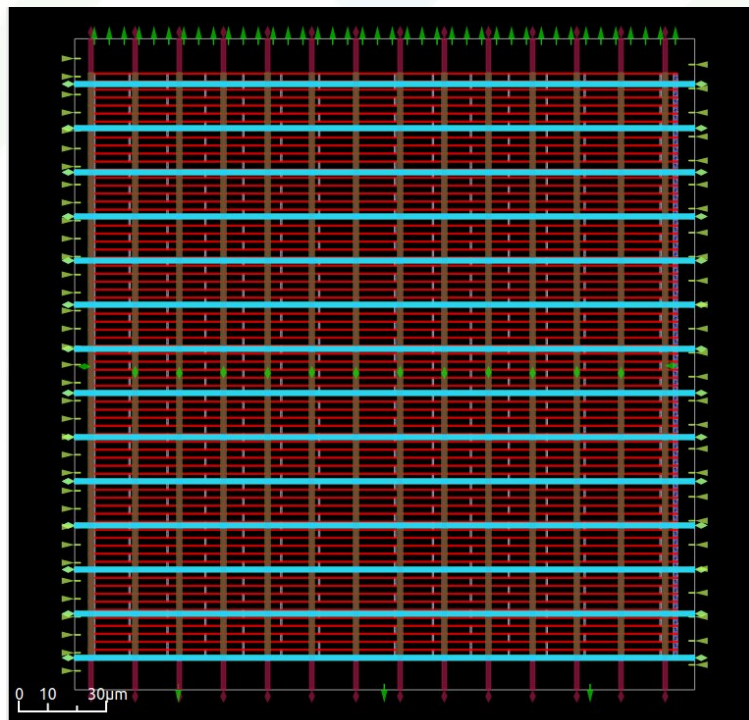


Figure 14: Combined power distribution network across all metal layers (`met1–met5`).

Placement and CTS Optimization

Routing Configuration: DRT_OPT_ITERS Tradeoff

The detailed router's optimization iteration count (`DRT_OPT_ITERS`) controls how many optimization passes `TritonRoute` performs to resolve routing violations, directly trading off runtime against the final DRC violation count. To characterize this trade-off, a design exploration sweep was performed using OpenLane's `run_designs.py` utility across a range of `DRT_OPT_ITERS` values, with the resulting routing runtime and DRC violation count recorded for each trial, as summarized in Table 7.

Iterations	Run Time	DRCs
64	9min, 12sec	No violations
48	8min, 47sec	No violations
32	8min, 20sec	No violations
20	8min, 05sec	No violations
10	6min, 34sec	6 violations
6	5min, 47sec	19 violations
4	4min, 58sec	95 violations
2	3min, 51sec	281 violations
1	3min, 05sec	412 violations

Table 7: Detailed routing optimization iterations vs. runtime and DRC violation count.

The results in Table 7 show a clear inflection point around 20 iterations: down to this value, the detailed router still converges to a fully clean layout with zero DRC violations, while runtime drops modestly as the iteration count is reduced. Below 20 iterations, however, violations start to appear and escalate sharply, climbing into the hundreds by the time the iteration count drops to 1–2, even though the runtime savings at that point are relatively small compared to the increase in violation count. This indicates that most of the router's optimization effort is genuinely needed to resolve routing conflicts, and that aggressively cutting iterations trades away DRC cleanliness for very little runtime benefit. Based on this exploration, `DRT_OPT_ITERS = 64` (the tool default) was retained for the final flow, as the extra runtime over the lowest clean value (20 iterations) is minor, while providing additional margin against DRC violations across different design configurations.

GDS and LEF Comparison in KLayout

To verify consistency between the physical implementation and its abstracted view, the final GDS (`wbqspiflash.klayout.gds`) and the corresponding LEF (`wbqspiflash.lef`) were opened side by side in KLayout and compared.

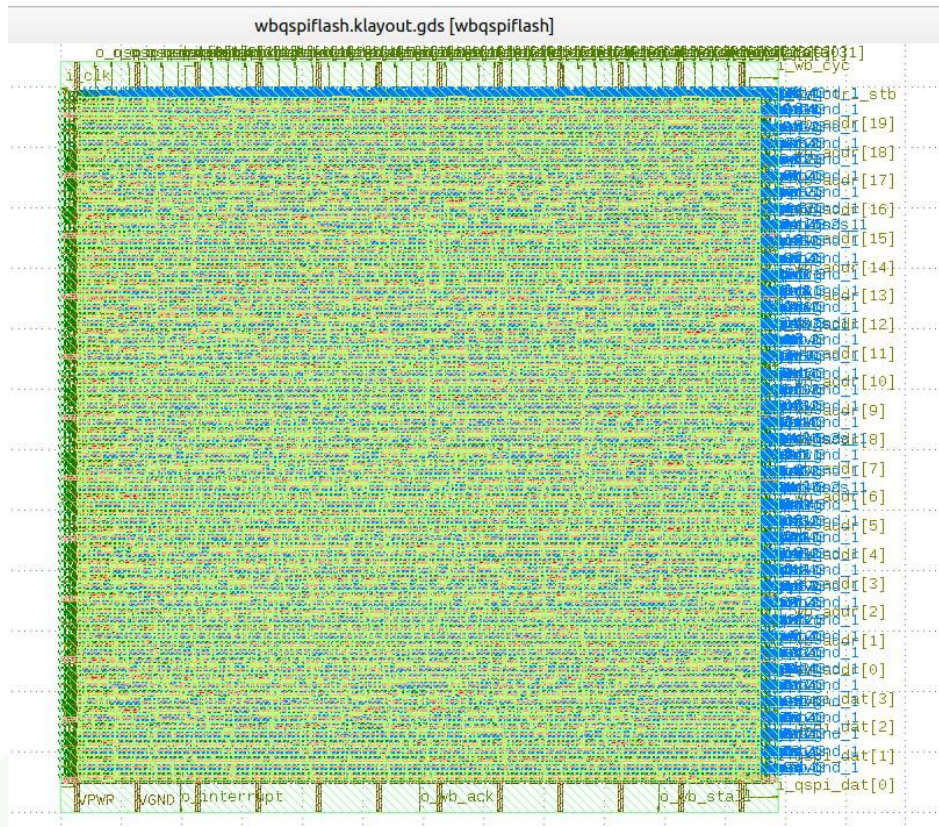


Figure 15: Full-chip view of the GDS layout showing all metal routing layers, standard cell rows, and I/O pins.

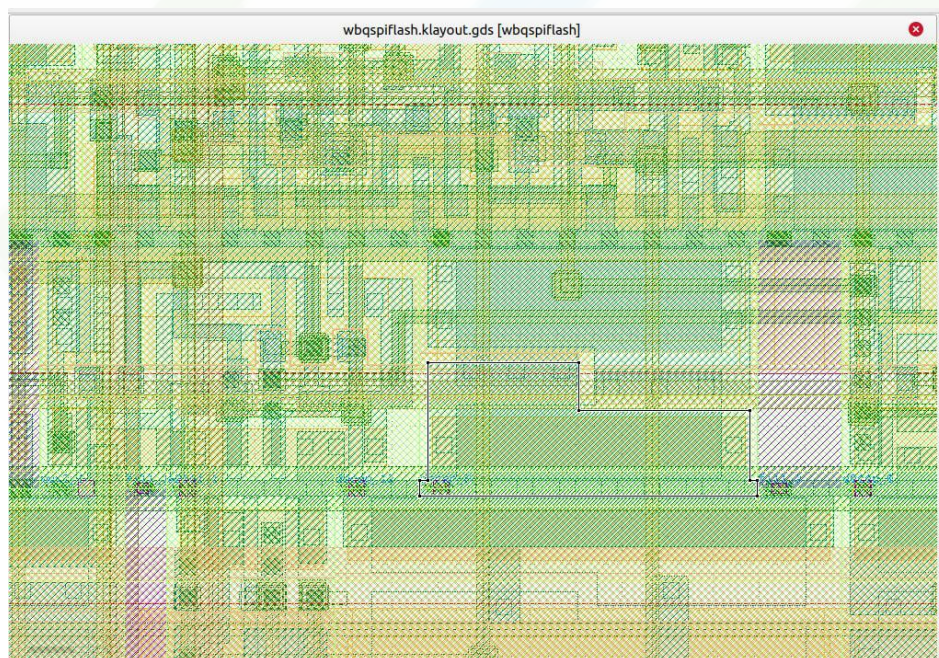


Figure 16: Zoomed-in view of the GDS layout showing detailed routing across multiple metal layers.

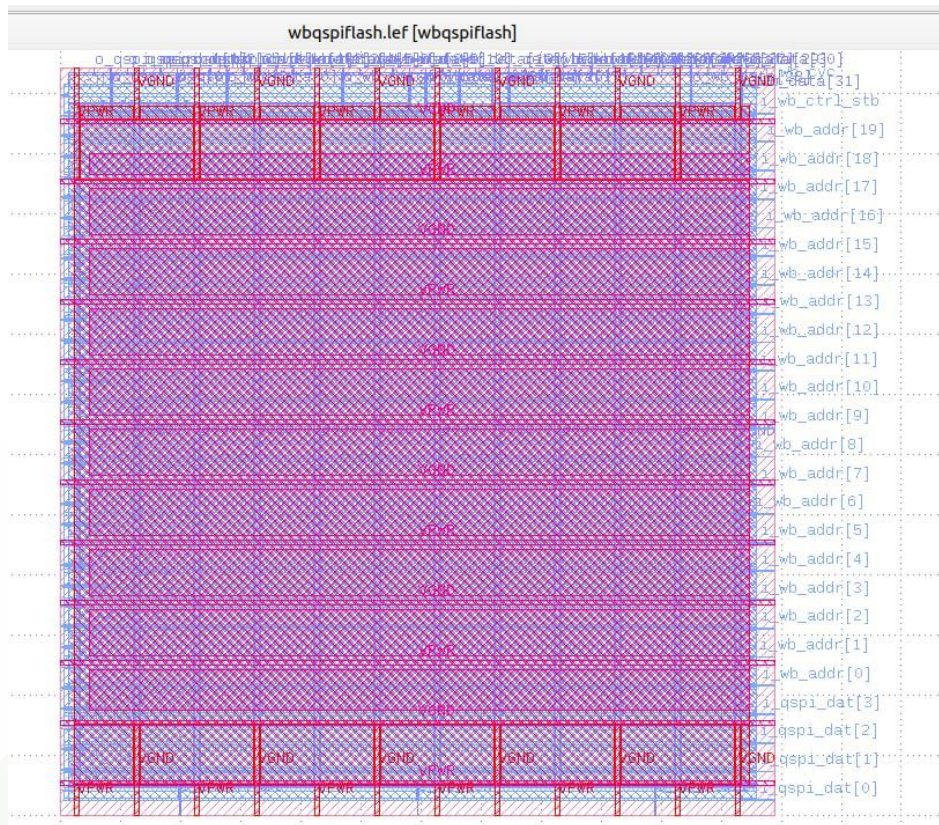


Figure 17: Full macro view of the LEF abstraction showing pin locations and power rail geometry only.

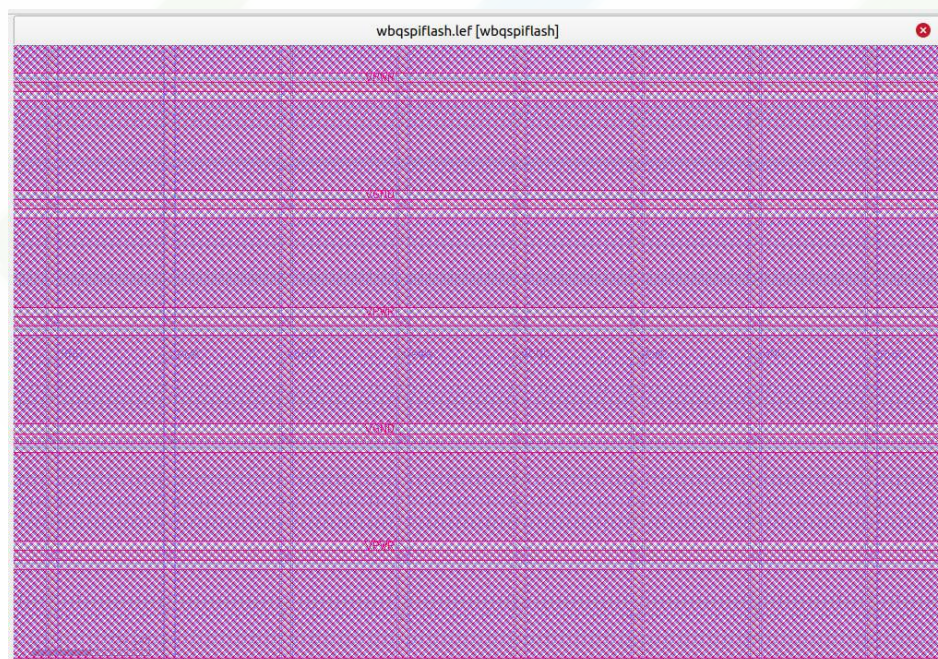


Figure 18: Zoomed-in view of the LEF abstraction, showing only the repeating VPWR/VGND power straps.

Observed Differences

Comparing the GDS and LEF views in Figures 15–18 shows the following key differences:

- **Level of detail:** The GDS contains the full mask-level geometry – all standard cells, transistors, and routing across every metal layer – while the LEF shows only the power straps (VPWR/VGND) and boundary pins, with all internal placement and routing hidden.
- **Purpose:** The GDS is the actual layout used for fabrication, whereas the LEF is a lightweight abstraction that lets the block be treated as a black box during higher-level place-and-route.
- **Pin consistency:** All I/O pins (`wb_addr[*]`, `wb_dat[*]`, `qspi_dat[*]`, `o_wb_ack`, `o_wb_stall`, `o_interrupt`, VPWR, VGND) match in name and position between the two views, confirming a correct LEF extraction.
- **File size:** This difference in detail is reflected directly in file size – the LEF is only **34.0 kB**, while the full GDS is **3.4 MB**, roughly **100×** larger, since the GDS encodes every physical shape in the design while the LEF encodes only pins and power geometry.

Overall, the LEF is confirmed to be a correct and complete abstraction of the GDS: pin and power information is preserved, while internal implementation detail is intentionally stripped out, resulting in a much smaller file suited for hierarchical integration.

Overall, the comparison confirms that the LEF is a correct and complete abstraction of the GDS: all boundary pins and power connections are preserved, while internal implementation detail is – as expected – intentionally omitted.

Metal Layer Validation in KLayout

To validate which metal layers are used for pin creation, the final GDS (`wbqspiflash.klayout.gds`) was opened in KLayout together with the technology's layer-mapping (`.map`) file.

Layer Mapping File

The `.map` file defines, for each metal, a wildcard rule linking GDS layer numbers to datatypes, as shown in Figure 19. Each metal is assigned a fixed GDS layer number, with datatype 5 reserved for text labels and datatype 44 reserved for vias, while all other datatypes correspond to drawing/pin shapes on that metal:

- `m2_wildcard = "69/0-4,6-43,45-*" → met2 = GDS layer 69`
- `m3_wildcard = "70/0-4,6-43,45-*" → met3 = GDS layer 70`

```
# layers definitions
#####

# all except purpose (datatype) 5 -- label and 44 -- via
li_wildcard = "67/0-4,6-43,45-*"
mcon_wildcard = "67/44"

m1_wildcard = "68/0-4,6-43,45-*"
via_wildcard = "68/44"

m2_wildcard = "69/0-4,6-43,45-*"
via2_wildcard = "69/44"

m3_wildcard = "70/0-4,6-43,45-*"
via3_wildcard = "70/44"

m4_wildcard = "71/0-4,6-43,45-*"
via4_wildcard = "71/44"

m5_wildcard = "72/0-4,6-43,45-*"
```

Figure 19: Layer definitions file showing the GDS layer number assigned to each metal (met1 through met5) and their via/label datatypes.

Pin Validation in KLayout

Isolating layer **69/20 (met2)** in KLayout shows the vertical I/O pins running along the top and bottom edges of the macro, as shown in Figure 20.

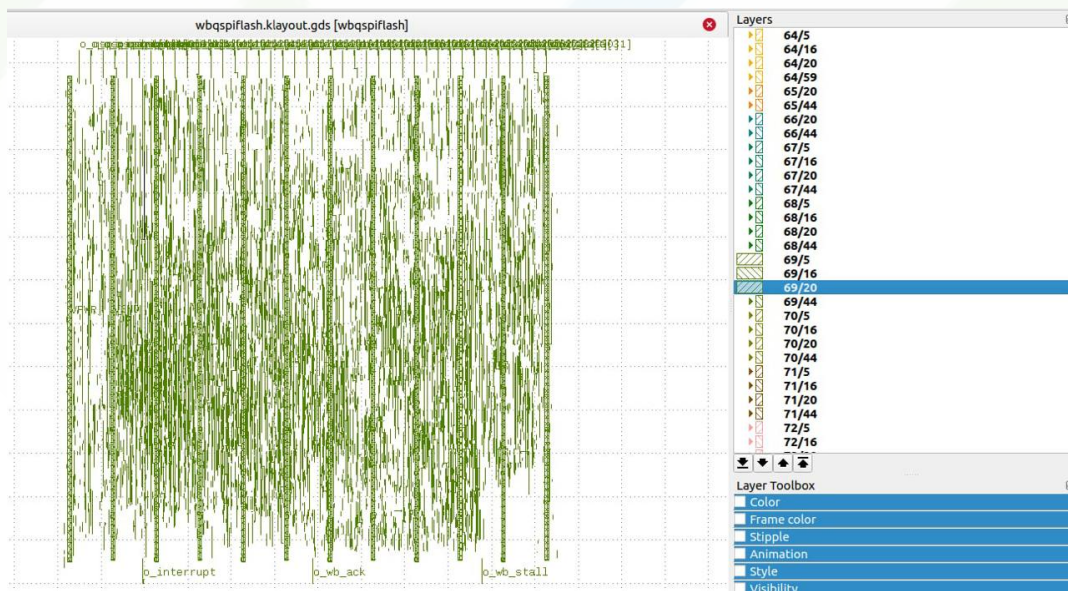


Figure 20: GDS view with only layer 69/20 (met2) enabled, showing the vertical pin shapes for signals such as VPWR, o_interrupt, o_wb_ack, and o_wb_stall.

Similarly, isolating layer **70/20 (met3)** shows the horizontal I/O pins running along the left and right edges of the macro, as shown in Figure 21.

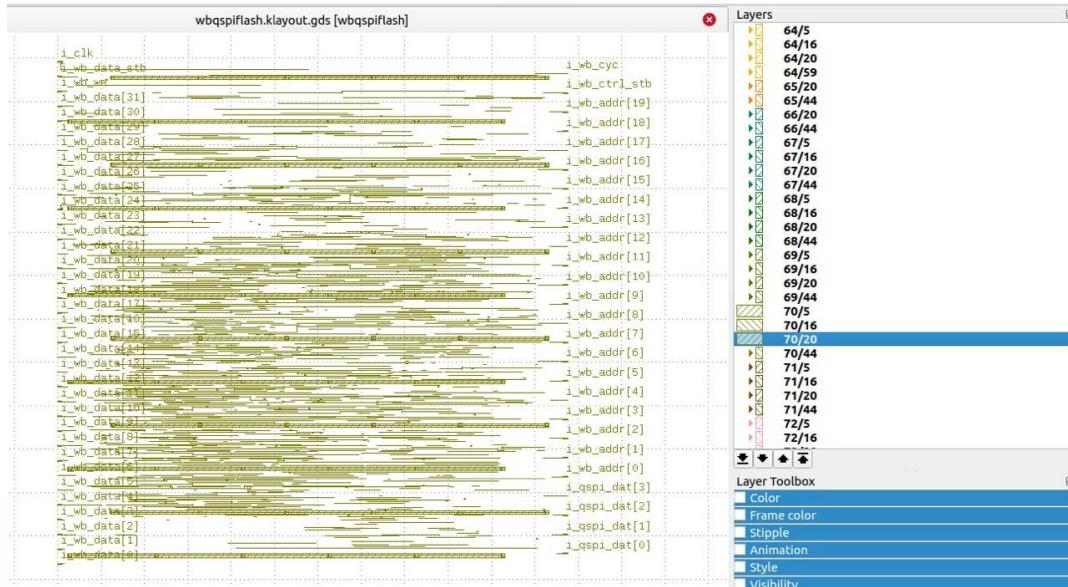


Figure 21: GDS view with only layer 70/20 (met3) enabled, showing the horizontal pin shapes for signals such as `i_wb_data[*]`, `i_wb_addr[*]`, and `i_qspi_dat[*]`.

This result directly matches the floorplan pin configuration used during synthesis/place-and-route, where the vertical and horizontal I/O layers are explicitly set as:

```
env(FP_IO_VLAYER) {met2}
env(FP_IO_HLAYER) {met3}
```

Confirming in KLayout that all vertical pins lie exclusively on layer 69 (met2) and all horizontal pins lie exclusively on layer 70 (met3) verifies that the pin placement configuration was correctly applied during the physical implementation of the design.

LEF Dimension Validation

To validate that the macro's abstracted boundary correctly reflects the physical die area established during floorplanning, the LEF file (`wbqspiflash.lef`) was opened in KLayout and its OUTLINE layer dimensions were measured, as shown in Figure 22.

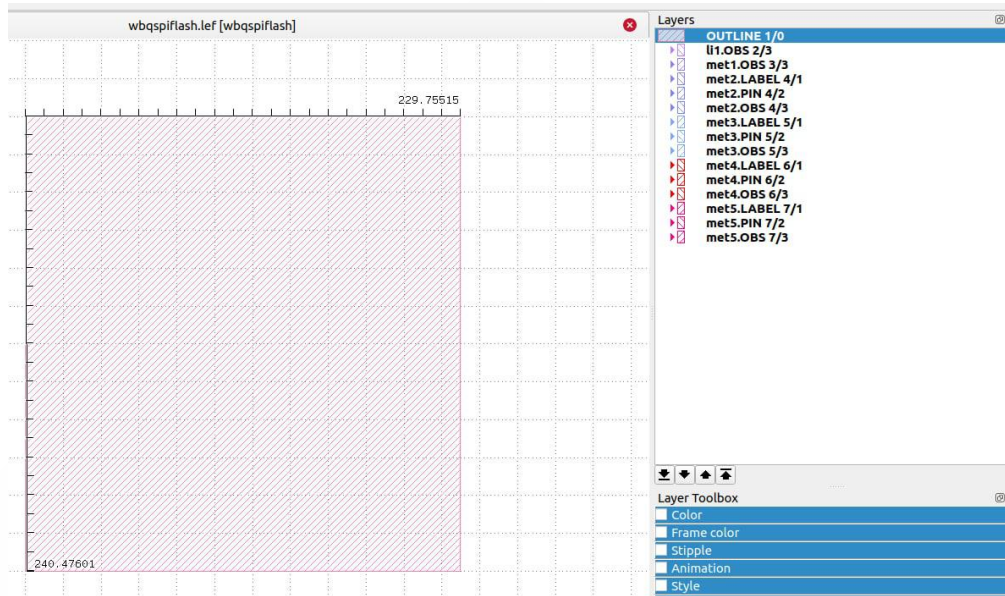


Figure 22: LEF outline of `wbqspiflash` in KLayout, showing the macro boundary spanning from (0,0) to (229.755, 240.476), along with the metal PIN/OBS/LABEL layer definitions for met1 through met5.

The die area reported by the floorplanning stage was:

$$(0.0, 0.0) \rightarrow (229.755, 240.475)$$

The LEF `OUTLINE` layer (1/0) measured directly in KLayout spans:

$$(0.0, 0.0) \rightarrow (229.755, 240.476)$$

Comparing the two, the LEF boundary matches the floorplan die area to within $0.001\mu\text{m}$ in the Y-dimension, a negligible difference attributable to rounding/snapping to the manufacturing grid during LEF generation. This confirms that the final LEF abstraction accurately preserves the exact die dimensions defined during floorplanning, with no unintended shrinkage, offset, or expansion of the macro boundary.

Full Routed Chip

The figure below shows the fully routed layout of the final chip, with the standard-cell rows visible in the core area, the horizontal power/ground straps running across the design, and the I/O pin connections extending outward from the core boundary on all four sides.

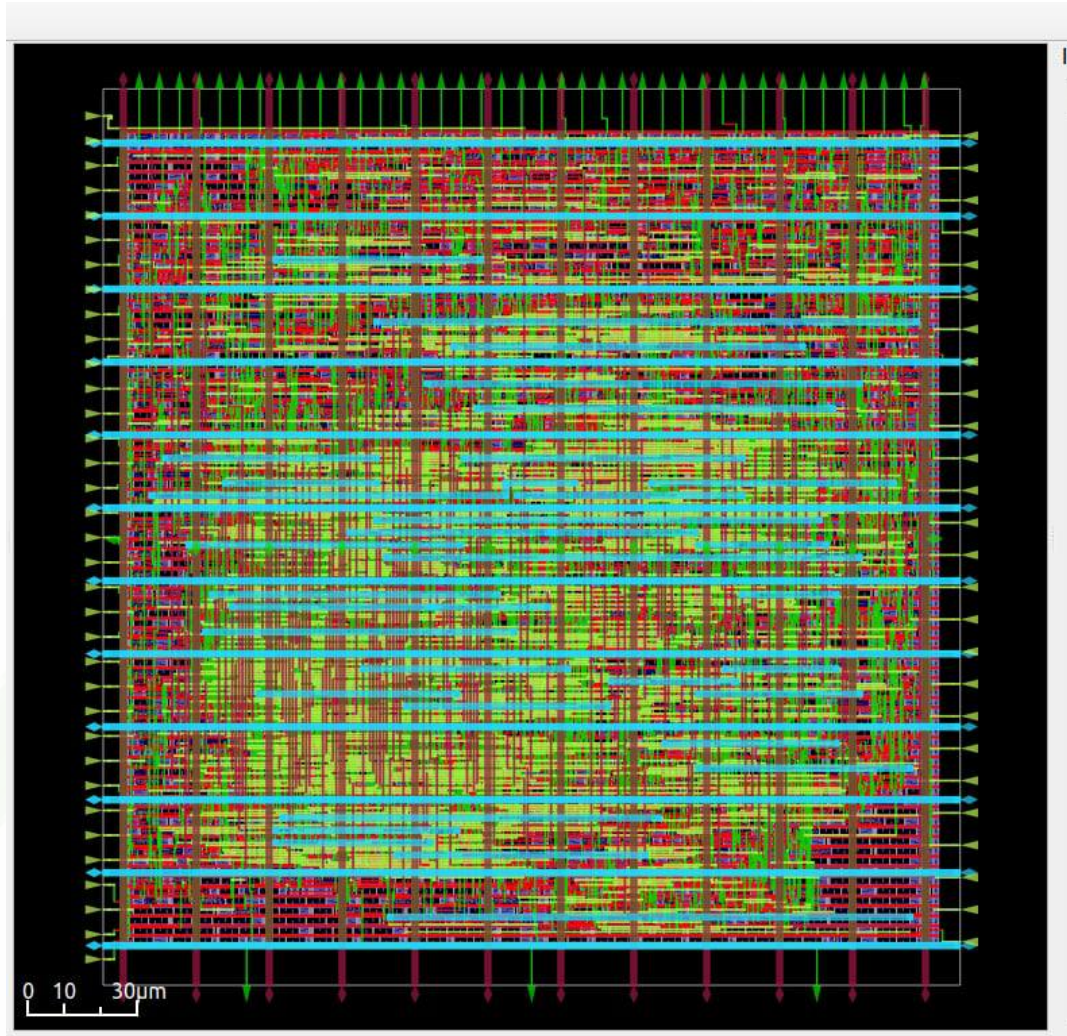


Figure 23: Fully routed layout of the final chip, showing standard-cell placement, power/ground straps, and I/O pin connections.

Extra Automation Section

Beyond the design exploration scripting described earlier, several additional Python-based automation environments were developed to streamline the day-to-day PnR workflow: a **report parsing environment**, a **log parsing environment**, a **liberty (.lib) parsing function**, and a **gate-level netlist parsing function**, all relying extensively on Python's **re** (regular expression) library. This automation environment is available in my own repository: [Python-ASIC-Flowscope](#).

Figure 24 shows the overall file hierarchy of this automation environment, organized into three main directories: **in_out** for the gate-level netlist I/O extraction, **log** containing the per-stage log parsing scripts, and **reports** containing the per-stage report summarization scripts, the NAND2 area utility (`nand_area.py`), and top-level runner scripts (`run_all_reports.py`, `run_all_log_parsers.py`) that invoke every stage script in sequence.

```
Thu Sep 10 03:46:49 PM EEST 2026
Ahmed-Alaa@home/icpedia/ahmed_alaa/python> ls -R
.:
in_out log reports

./in_out:
in_out in_out_extraction.py in_out.txt

./log:
cts_log_parasing.py      routing_log_parasing.py  syn_log_parsing.py
floorplan_log_parasing.py run_all_log_parsers.py
placement_log_parasing.py signoff_log_parasing.py

./reports:
nand_area.py      test_cts.py      test_routing.py
__pycache__      test_floorplan.py test_signoff.py
run_all_reports.py test_placement.py test_syn.py

./reports/__pycache__:
nand_area.cpython-310.pyc
```

Figure 24: File hierarchy of the automation environment, showing the `in_out`, `log`, and `reports` directories and their respective scripts.

Integration into flow.tcl

To integrate this automation environment directly into the OpenLane flow, `flow.tcl` was modified to automatically invoke the report and log parsing scripts at the end of each run. Two new procedures, `run_all_reports_step` and `run_all_logs_step`, were added to call `run_all_reports.py` and `run_all_log_parsers.py` respectively, passing the current `REPORTS_DIR` and `RUN_DIR/logs` paths as arguments (Figure 25). These steps were then hooked into the main flow sequence, right after `generate_final_summary_report` and before `check_timing_violations`, ensuring the parsing scripts run automatically once the flow completes (Figure 26).

```

227     }
228
229     if { [ string equal $arg_values(-to) $step_name ] } {
230         set exe 0:
231         break;
232     }
233
234 }
235
236
237 # ADDED PART
238 proc run_all_reports_step {args} {
239     puts_info "Running report summary scripts..."
240     set reports_path $::env(REPORTS_DIR)
241     set result [exec python3 /home/icpedia/ahmed_alaa/python/reports/run_all_reports.py $reports_path]
242     puts_info "$result"
243 }
244
245 proc run_all_logs_step {args} {
246     puts_info "Running log parsing scripts..."
247     set logs_path $::env(RUN_DIR)/logs
248     set result [exec python3 /home/icpedia/ahmed_alaa/python/log/run_all_log_parsers.py $logs_path]
249     puts_info "$result"
250 }
251
252 # for when it resumes
253 set steps_as_list [dict keys $steps]
254 set next_idx [expr [lsearch $steps_as_list $::env(CURRENT_STEP)] + 1]
255 set ::env(CURRENT_STEP) [lindex $steps_as_list $next_idx]
256
257 # Saves to <RUN_DIR>/results/final
258 save_final_views
259
260 # Saves to design directory or custom
261 if { [info exists flags_map(-save)] } {
262     if { ! [info exists arg_values(-save_path)] } {
263         set arg_values(-save_path) $::env(DESIGN_DIR)
264     }
265     save_final_views \
266         -save_path $arg_values(-save_path) \
267         -tag $::env(RUN_TAG)
268 }
269
270 calc total runtime

```

flow.tcl

Figure 25: Added procedures `run_all_reports_step` and `run_all_logs_step` in `flow.tcl`, invoking the Python report and log parsing scripts.

```

256 # Saves to <RUN_DIR>/results/final
257 save_final_views
258
259 # Saves to design directory or custom
260 if { [info exists flags_map(-save) ] } {
261     if { ! [info exists arg_values(-save_path)] } {
262         set arg_values(-save_path) $::env(DSIGN_DIR)
263     }
264     save_final_views\
265         -save_path $arg_values(-save_path)\
266         -tag $::env(RUN_TAG)
267 }
268 calc_total_runtime
269 save_state
270 generate_final_summary_report
271 # added part #####
272 run_all_reports_step
273 run_all_logs_step
274 #####
275 check_timing_violations
276
277 if { [info exists arg_values(-save_path)]\
278     && $arg_values(-save_path) != "" } {
279     set ::env(HOOK_OUTPUT_PATH) "[file normalize $arg_values(-save_path)]"
280 } else {
281     set ::env(HOOK_OUTPUT_PATH) $::env(RESULTS_DIR)/final
282 }
283
284 if {[info exists flags_map(-run_hooks)]} {
285     run_post_run_hooks
286 }
287
288 puts_success "Flow complete."
289
290 show_warnings "Note that the following warnings have been generated:"
291 }
292
293 proc run_interactive_mode {args} {
294     set options {
295         {-design optional}
296     }
297     set flags {}

```

Figure 26: Calls to `run_all_reports_step` and `run_all_logs_step` hooked into the main flow sequence in `flow.tcl`.

Report Summarization

Each stage of the OpenLane flow – synthesis, floorplanning, placement, clock tree synthesis, routing, and sign-off – generates a large number of individual report files (timing, power, area, slew, skew, violation counts, and so on). Manually opening each of these files after every run to check for issues or compare metrics is time-consuming and error-prone. To address this, a dedicated Python script was written for each stage that parses all of the relevant reports for that stage using regular expressions, extracts the key metrics (area, utilization, WNS/TNS, worst setup/hold slack, clock skew, power, slew and fanout violation counts, DRC/XOR results, etc.), and consolidates them into a single, readable summary report.

These scripts were integrated directly into OpenLane's `flow.tcl`, so that the appropriate summary is generated automatically at the end of each stage without requiring the functions to be invoked manually. The scripts were also written generically, taking the run's reports directory as a command-line argument rather than hard-coding paths for this specific design, so the same automation can be reused for any OpenLane design without modification.

Figure 27 shows the terminal output at the end of a full flow run, confirming that all five summary scripts (`test_syn.py`, `test_floorplan.py`, `test_placement.py`, `test_cts.py`, `test_routing.py`, and `test_signoff.py`) are triggered automatically by `flow.tcl` immediately after the corresponding stage completes, with each script also printing the NAND2 area retrieved from the liberty file before writing its summary report to the appropriate `reports/` subdirectory.

```
[INFO]: Running report summary scripts...
[INFO]: =====
Running: test_syn.py
=====
sky130_fd_sc_hd_nand2_1 -> area = 3.7536000000
Summary written to /home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/reports/synthesis/synthesis_summary.rpt
=====
Running: test_floorplan.py
=====
Summary report written to: /home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/reports/floorplan/floorplan_summary.rpt
=====
Running: test_placement.py
=====
sky130_fd_sc_hd_nand2_1 -> area = 3.7536000000
Summary report written to: /home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/reports/placement/placement_summary.rpt
=====
Running: test_cts.py
=====
sky130_fd_sc_hd_nand2_1 -> area = 3.7536000000
Summary report written to: /home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/reports/cts/cts_summary.rpt
=====
Running: test_routing.py
=====
sky130_fd_sc_hd_nand2_1 -> area = 3.7536000000
Summary report written to: /home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/reports/routing/routing_summary.rpt
=====
Running: test_signoff.py
=====
sky130_fd_sc_hd_nand2_1 -> area = 3.7536000000
Summary report written to: /home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/reports/signoff/sign_off_summary.rpt
=====
```

Figure 27: Terminal log showing the report summary scripts being invoked automatically by `flow.tcl` at the end of each stage, with no manual intervention required.

Figure 28 shows the synthesis stage summary, reporting a design area of $20\,570\,\mu\text{m}^2$ (5480.07 K-NAND2 units) with zero setup and hold violations, a worst setup slack of 2.73 ns, a worst hold slack of 0.36 ns, and no max slew violations. Total power at this stage is reported as $3.05 \times 10^{-3}\text{ W}$.

```
=====
                          SYNTHESIS QoR SUMMARY REPORT
=====
Generated on : 2026-09-10 15:13:29
=====

1. AREA
-----
Design Area           : 20570 um^2
Area (K-NAND2 units)  : 5480.07

2. TIMING
-----
Worst Negative Slack (WNS) : 0.00 ns
Total Negative Slack (TNS) : 0.00 ns
Worst Setup Slack         : 2.73 ns
Worst Hold Slack          : 0.36 ns
Setup Violations           : No
Hold Violations            : No
Max Slew Violations        : 0

3. POWER
-----
Total Power            : 3.05e-03 W

=====
                          END OF REPORT
=====
```

Figure 28: Automatically generated synthesis stage summary report for the final selected trial (6.422 ns, 41% utilization), showing design area, timing slacks, and total power.

To illustrate the output of these scripts in practice, Figures 29–33 show the summary reports generated at each stage for the final selected run (`T_6.422_UTI41_0.45`), corresponding to the maximum achievable frequency (6.422 ns) and minimum area configuration (41% utilization, 0.45 target density) identified earlier in this chapter.

Figure 29 shows the floorplan stage summary, reporting a core area of $49\,862.82\,\mu\text{m}^2$ within a die area of

57 756.22 μm^2 , giving a core-to-die ratio of 86.33%. The report also summarizes the power delivery network quality, with a worst-case IR drop of only 5.61×10^{-10} V at a supply voltage of 1.80 V, alongside the pin, endcap, and tapcell counts (104 pins, 164 endcaps, and 714 tapcells) placed during floorplanning.

```
Ahmed-Alaa@/home/opentools/OpenLane/designs/wbqspiflash/runs/T_6.422_UTI41_0.45/reports/cts> cat floorplan_summary.rpt
```

Floorplan Summary Report - Generated 2026-09-10 15:13:30.049030	
Metric	Value
Core Area (μm^2)	49862.8224
Die Area (μm^2)	57756.218625
Core-to-Die Ratio (%)	86.33%
Power Delivery Network (IR Drop)	
Worstcase Voltage (V)	1.80e+00
Average IR Drop (V)	4.29e-10
Worstcase IR Drop (V)	5.61e-10
Pins, Endcaps & Tapcells	
Number of Pins	104
Number of Endcaps	164
Number of Tapcells	714

Figure 29: Automatically generated floorplan stage summary report for the final selected trial (6.422 ns, 41% utilization), showing core/die area, IR drop, and pin/endcap/tapcell counts.

Figure 30 shows the placement stage summary, comparing metrics before and after resize optimization, with the design area reduced to 20 268 μm^2 at 41% utilization with zero timing violations after optimization.

```
Ahmed-Alaa@/home/opentools/OpenLane/designs/wbqspiflash/runs/T_6.422_UTI41_0.45/reports/cts> cat placement_summary.rpt
```

Placement Timing Summary Report - Generated 2026-09-10 15:13:30.269189		
Metric	Before Opt (GPL)	After Opt (Resize)
Global Clock Skew	0.00	0.00
Setup Violation	No Violation	No Violation
Hold Violation	No Violation	No Violation
STA Violation	No Violation	No Violation
TNS	0.00	0.00
WNS	0.00	0.00
Post-Optimization Only Metrics		
Design Area (μm^2)	: 20268	
Utilization (%)	: 41	
Area (K-NAND2 units)	: 5399.62	
Total Power (Leakage, W)	: 2.99e-03	
Worst Setup Slack	: 2.45	
Worst Hold Slack	: 0.35	
Max Slew Violations	: 0	
Max Fanout Violations	: 0	
Max Cap Violations	: 0	

Figure 30: Automatically generated placement stage summary report for the final selected trial (6.422 ns, 41% utilization).

Figure 31 shows the CTS stage summary, confirming a clean clock tree with zero setup/hold violations and a global clock skew of 0.11 ns both before and after post-CTS optimization.

```

Thu Sep 10 04:29:25 PM EEST 2026
Ahmed-Alaa@home/opentools/OpenLane/designs/wbqspiflash/runs/T_6.422_UTI41_0.45/reports/cts> cat cts_summary.rpt
=====
CTS Timing Summary Report - Generated 2026-09-10 15:13:30.500289
=====

```

Metric	Before Opt (CTS)	After Opt (Resize)
Global Clock Skew	0.11	0.11
Setup Violation	No Violation	No Violation
Hold Violation	No Violation	No Violation
STA Violation	No Violation	No Violation
TNS	0.00	0.00
WNS	0.00	0.00

```

=====
Post-Optimization Only Metrics
=====
Design Area (um^2)      : 21344
Utilization (%)         : 43
Area (K-NAND2 units)    : 5686.27
Total Power (leakage, W) : 4.79e-03
Worst Setup Slack       : 3.08
Worst Hold Slack        : 0.32
Max Slew Violations     : 0
Max Fanout Violations   : 0

```

Figure 31: Automatically generated CTS stage summary report for the final selected trial (6.422 ns, 41% utilization).

Figure 32 shows the routing stage summary, including the detailed wire length breakdown across all metal layers (met1–met5) and the total via count following the resize and global route (GRT) steps, both of which report zero timing violations.

```

Overall Status: PASS
Violations Found: None

```

Metric	Step 15 (Resize)	Step 18 (GRT)
Global Clock Skew	0.12	0.12
Setup Violation	No Violation	No Violation
Hold Violation	No Violation	No Violation
STA Violation	No Violation	No Violation
TNS	0.00	0.00
WNS	0.00	0.00

```

=====
Step 15 (Resize) Only Metrics
=====
Design Area (um^2)      : 21344
Utilization (%)         : 43
Area (K-NAND2 units)    : 5686.27
Total Power (leakage, W) : 4.88e-03
Max Slew Violations     : 0
Max Fanout Violations   : 0
Max Cap Violations      : 0

```

```

=====
Detail Routing Summary
=====
Total Wire Length (um)   : 86607
Wire Length on li1       : 0 um
Wire Length on met1      : 23571 um
Wire Length on met2      : 31683 um
Wire Length on met3      : 19791 um
Wire Length on met4      : 8217 um
Wire Length on met5      : 2454 um
Total Number of Vias     : 17822

```

Figure 32: Automatically generated routing stage summary report for the final selected trial (6.422 ns, 41% utilization), including per-layer wire length and via count.

Finally, Figure 33 shows the sign-off stage summary, confirming that the final design closes cleanly across all MCA corners (Slowest, Typical, Fastest) with no setup or hold violations, zero DRC violations, and a final design area of $48\,269\,\mu\text{m}^2$ (12859.39 K-NAND2 units) at 97% utilization.

```

=====
Signoff Summary Report - Generated 2026-09-10 15:13:32.784164
=====

-----
MCA Corner Details
-----
Metric                Slowest      Typical      Fastest
Setup Violation        No Violation No Violation No Violation
Hold Violation          No Violation No Violation No Violation
Clock Skew              0.20        0.11        0.08
Total Power             3.78e-03    4.78e-03    5.54e-03

-----
Other Signoff Metrics
-----
Design Area (um^2)      : 48269
Utilization (%)         : 97
Area (K-NAND2 units)    : 12859.39
Total Power (rcx_sta, W) : 4.78e-03
TNS (rcx_sta)           : 0.00
WNS (rcx_sta)           : 0.00
TNS (MCA)               : 0.00
WNS (MCA)               : 0.00
Worst Setup Slack (rcx_sta) : 3.07
Worst Hold Slack (rcx_sta) : 0.33
Worst Setup Slack (MCA)   : 0.06
Worst Hold Slack (MCA)   : 0.17
Max Slew Violations      : 0
Max Fanout Violations    : 0
Max Cap Violations       : 0
XOR Differences          : 104
DRC Violation Count      : 0

```

Figure 33: Automatically generated sign-off stage summary report for the final selected trial (6.422 ns, 41% utilization), showing clean timing across all MCA corners and zero DRC violations.

Log Parsing

A second, complementary environment was built to parse the raw tool logs produced throughout the flow. Rather than summarizing metrics, this script scans each log file for error-level messages using regular expressions and, whenever a match is found, prints out the offending line along with its source log file. This gives a quick, centralized way to confirm that a run completed cleanly, or to immediately locate the source of a failure, without manually scrolling through dozens of log files.

Similarly to the report summarization scripts, a dedicated log parsing script was written for each stage (`syn_log_parsing.py`, `floorplan_log_parsing.py`, `placement_log_parsing.py`, `cts_log_parsing.py`, `routing_log_parsing.py`, and `signoff_log_parsing.py`), and these were likewise integrated into `flow.tcl` to run automatically at the end of each stage. Figure 34 shows the terminal output of these scripts at the end of a full flow run: each stage's individual tool logs are checked in turn, with `[OpenROAD/Magic - <Stage>] <log file> No errors found.` printed for a clean log, or the offending line reported directly when an issue (such as the missing LEF/LVS files and DRC error flagged during the sign-off stage) is detected. This allows the entire flow to be checked for errors at a glance, without opening a single log file manually.

```

Running: floorplan_log_parsing.py
=====
[OpenROAD - Floorplan Stage] [3-initial.fp.log] No errors found.
[OpenROAD - Floorplan Stage] [4-place_io.ol.log] No errors found.
[OpenROAD - Floorplan Stage] [5-tap.log] No errors found.
[OpenROAD - Floorplan Stage] [6-pdn.log] No errors found.
=====
Running: placement_log_parsing.py
=====
[OpenROAD - Placement Stage] [7-global.log] No errors found.
[OpenROAD - Placement Stage] [8-resizer.log] No errors found.
[OpenROAD - Placement Stage] [9-write_verilog.log] No errors found.
[OpenROAD - Placement Stage] [10-detailed.log] No errors found.
=====
Running: cts_log_parsing.py
=====
[OpenROAD - CTS Stage] [11-cts.log] No errors found.
[OpenROAD - CTS Stage] [12-write_verilog.log] No errors found.
[OpenROAD - CTS Stage] [13-resizer.log] No errors found.
[OpenROAD - CTS Stage] [14-write_verilog.log] No errors found.
=====
Running: routing_log_parsing.py
=====
[OpenROAD - Routing Stage] No errors found in any routing log.
=====
Running: signoff_log_parsing.py
=====
[Magic - Signoff Stage] File not found: /home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/logs/signoff/36-spn.lef.log
[Magic - Signoff Stage] File not found: /home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/logs/signoff/36-spn.lvs.lef.log
[Magic - Signoff Stage] [37-drc.log] [Line 134] No errors found.
[Magic - Signoff Stage] [37-drc.log] [Line 138] [INFO]: Saving nag view with DRC errors (/home/opentools/OpenLane/designs/wbqspiflash/runs/full_flow_trial/results/signoff/wbqspiflash.drc.nag)
=====
All log parsing scripts finished running.
=====

```

Figure 34: Terminal output of the automated log parsing scripts, run stage by stage at the end of the flow, flagging errors where found and confirming clean logs otherwise.

Liberty (.lib) Parsing for Area Normalization

A dedicated function was also written to parse the sky130 standard cell liberty (.lib) file and extract the area of the basic 2-input NAND gate (`sky130_fd_sc_hd_nand2_1`), again using regular expressions to locate the cell definition and pull out its `area` field. This NAND2 area is used as a reference unit to express the design's total area in terms of **K-NAND2 equivalent gates** (design area divided by NAND2 area), a standard, technology-independent metric for reporting design size. This function was integrated into the report summarization scripts described above, so that every stage summary automatically includes the design area expressed in K-NAND2 units alongside the raw area in μm^2 .

Gate-Level Netlist Parsing

Finally, a function was written to parse the design's gate-level (post-synthesis) netlist and extract its primary inputs and outputs directly from the port declarations, again using regular expressions rather than manual inspection. This provides a fast way to verify the design's I/O interface at the netlist level, confirming that synthesis preserved the expected port list and that no pins were unintentionally added, removed, or renamed during the flow.

Figure 35 shows the output of this script when run on the `wbqspiflash` gate-level netlist, correctly identifying all input and output ports along with their bus widths (e.g. `i_wb_addr[19:0]`, `o_wb_data[31:0]`), and cleanly separating them into two labeled INPUTS/OUTPUTS sections.

```
Ahmed-Alaa@/home/icpedia/ahmed_alaa/python/in_out> cat in_out.txt
INPUTS:
input i_clk;
input [3:0] i_qspi_dat;
input [19:0] i_wb_addr;
input i_wb_ctrl_stb;
input i_wb_cyc;
input [31:0] i_wb_data;
input i_wb_data_stb;
input i_wb_we;

OUTPUTS:
output o_interrupt;
output o_qspi_cs_n;
output [3:0] o_qspi_dat;
output [1:0] o_qspi_mod;
output o_qspi_sck;
output o_wb_ack;
output [31:0] o_wb_data;
output o_wb_stall;
```

Figure 35: Example output of the gate-level netlist I/O extraction script, showing all input and output ports of the `wbqspiflash` design.

Results Summary

Two operating points were explored to characterize the trade-off between speed and area for this design:

- **Maximum frequency:** A clock period of 6.422 ns was reached, corresponding to 155.71 MHz ($1/6.422 \text{ ns} \approx 155.71 \text{ MHz}$). At this frequency, the minimum achievable design area was $48\,269 \mu\text{m}^2$.
- **Relaxed frequency:** With the timing constraint relaxed to a clock period of 10 ns (100 MHz), the design reached a significantly smaller minimum area of $30\,234 \mu\text{m}^2$.

These results illustrate the expected inverse relationship between operating frequency and achievable area: pushing for higher frequency (155.71 MHz) requires a larger area ($48\,269 \mu\text{m}^2$) to meet timing, while relaxing the frequency to 100 MHz allows the area to shrink to $30\,234 \mu\text{m}^2$.

Conclusion

This project implemented a complete ASIC backend flow using OpenROAD and OpenLane, from synthesis through place-and-route to sign-off, supported by a custom automation environment built around it.

Design and synthesis exploration across frequency and utilization/density settings made it possible to systematically map the trade-off between achievable clock frequency and design area, rather than relying on a single manual run. Automated Python scripts for parsing logs and reports at each stage, integrated directly into `flow.tcl`, saved significant time by removing the need to manually inspect report files and making it easy to trace and compare results across many runs. Running the flow with multiple threads further sped up this exploration, allowing far more configurations to be tested than a sequential run would allow.

Overall, this exploration methodology combined with the automation environment made it possible to efficiently converge on a final design that meets timing with minimal area. Claude was also used throughout the project to help develop the parsing scripts and draft parts of this report.